

VibeMeGood — Complete Implementation Plan

Single-user private analytics workstation. Everything from start to finish.

Hand this document and all attachments to your Replit agent. Read this first. Then read the referenced files. Then build.

CONTEXT

This is a private, single-user decision intelligence terminal built on top of PrizePicks. It is not a public platform. Community features, social sharing, leaderboards, and copy-trading are irrelevant. Every feature exists for one purpose: make one person's picks systematically better over time.

The primary long-term proof metric: **CLV trending positive over 90+ days.**

ATTACHED FILES — WHAT EACH ONE IS

File	Purpose	Give to Agent?
VibeMeGood-main.zip	Existing codebase — build on top of this	✓ Yes
VibeMeGood-Instructions-Final.md	Core build mandate + acceptance tests	✓ Yes
VibeMeGood-ReplitSpec.md	Full implementation reference with schemas and code	✓ Yes
VibeMeGood-QC-Fixes.md	10 confirmed bugs found in QC — fix these first	✓ Yes
VibeMeGood-VarianceEngine.md	Contextual variance engine — toggle-based add-on	✓ Yes
VibeMeGood-PlayerPhotos.md	Player thumbnail photos — simple visual upgrade	✓ Yes

VibeMeGood-MasterPlan.md	This file — read first	✓ Yes
VibeMeGood-UserGuide.md	User guide content — embed in the app at /guide	✓ Yes

CURRENT STATE OF THE BUILD

The app has been partially built by a previous agent. Here is what is confirmed working and what is confirmed broken or missing.

✓ Already Working

- PrizePicks live sync with stale line deactivation
- External odds sync (DraftKings, FanDuel, ESPN Bet)
- Bayesian projection engine with shrinkage, P(over), standard deviation
- Market intelligence route with 4-state source health
- Server-Sent Events real-time connection
- Force Sync button (slate board + triggered from sidebar)
- All 12 pages routed: dashboard, slate, injuries, entry-builder, journal, review, ai-chat, streaks, clv, matchup, guide, settings
- Entry context with max 6 picks, direction management
- EV calculation with proper flex dynamic programming
- CLV auto-recording on pick resolution
- Debug calibration endpoint at `/api/debug/calibration`
- Startup projection warm-up on server start
- Loss limit infrastructure
- Behavioral logging on entry submission
- AI analyst with real DB context injected

● Confirmed Broken — Fix Before Anything Else

See `VibeMeGood-QC-Fixes.md` for full detail on each. Summary:

1. Mobile Slate Board — 14 columns, no responsive hiding, unusable on phone
 2. Force Sync missing projections — `/sync/all` doesn't refresh projections
 3. No DB transactions on result resolution — partial failures corrupt history
 4. No Zod validation — bad input writes garbage instead of returning 400
 5. Auth soft — `req.query.userId` instead of Clerk middleware
 6. AI chat hardcoded base URL — breaks on subpaths
 7. Injuries sync is a stub — returns 0, page is empty
 8. Mark Results hard to find — no pointer in Entry Builder
 9. Slate Board mobile filter drawer missing
 10. Force Sync has no intermediate feedback
-

BUILD ORDER — FOLLOW THIS EXACTLY

Do not skip phases. Each phase depends on the previous.

PHASE 1: Fix What's Broken (Week 1)

Read: `VibeMeGood-QC-Fixes.md`

Fix all 10 confirmed bugs in priority order:

1. Mobile table responsive hiding
2. Force Sync includes projections
3. DB transactions on result resolution
4. Zod validation on all input routes
5. Clerk auth replacing `req.query.userId`
6. AI chat BASE URL
7. Injuries stub → real implementation (see Phase 3 for source)
8. Mark Results callout in Entry Builder
9. Mobile filter drawer on Slate Board

10. Force Sync intermediate SSE feedback

Acceptance: All 14 tests in `VibeMeGood-Instructions-Final.md` pass.

PHASE 2: Player Photos (Week 1 — same sprint as Phase 1)

Read: `VibeMeGood-PlayerPhotos.md`

Low complexity, high visual impact. Do this alongside the QC fixes.

1. Add `imageUrl` text column to players table
2. Capture `pAttr.image_url` in PP sync (2 lines)
3. Pass `imageUrl` through market-intel and slate routes
4. Add `imageUrl` to `EntryPick` type
5. Build `PlayerAvatar` component with initials fallback
6. Place in: Slate Board rows, Prop Detail Sheet, Entry Builder, Streak Tracker

Acceptance: Every player row shows a circular photo or colored initials. No broken image icons anywhere. Photos load lazily without blocking the UI.

PHASE 3: Injury Feed (Week 2)

The injury sync stub is the single biggest gap in the platform. Being first to know a player is scratched is worth more than any model improvement.

Data sources — all free, no key required:

```
// NBA: Official injury report via ESPN unofficial API
const NBA_INJURIES = "https://site.api.espn.com/apis/site/v2/sports/basketball/nb

// MLB injuries
const MLB_INJURIES = "https://site.api.espn.com/apis/site/v2/sports/baseball/mlb/

// NHL injuries
const NHL_INJURIES = "https://site.api.espn.com/apis/site/v2/sports/hockey/nhl/in

// NFL injuries
const NFL_INJURIES = "https://site.api.espn.com/apis/site/v2/sports/football/nfl/

// WNBA injuries
```

```
const WNBA_INJURIES = "https://site.api.espn.com/apis/site/v2/sports/basketball/w
```

ESPN injury response structure:

```
{
  "injuries": [
    {
      "athlete": {
        "displayName": "Anthony Davis",
        "team": { "abbreviation": "LAL" }
      },
      "status": "Questionable",
      "detail": "Left foot soreness",
      "date": "2026-05-23T14:00:00Z"
    }
  ]
}
```

Implementation — replace the stub in `sync.ts` :

```
export async function syncInjuriesImpl(): Promise<number> {
  const SPORT_URLS: Record<string, string> = {
    NBA: "https://site.api.espn.com/apis/site/v2/sports/basketball/nba/injuries"
    MLB: "https://site.api.espn.com/apis/site/v2/sports/baseball/mlb/injuries",
    NHL: "https://site.api.espn.com/apis/site/v2/sports/hockey/nhl/injuries",
    NFL: "https://site.api.espn.com/apis/site/v2/sports/football/nfl/injuries",
    WNBA: "https://site.api.espn.com/apis/site/v2/sports/basketball/wnba/injuries"
  };

  let processed = 0;

  for (const [sport, url] of Object.entries(SPORT_URLS)) {
    try {
      const res = await fetch(url, {
        headers: {
          "User-Agent": "Mozilla/5.0",
          "Accept": "application/json",
        }
      });
      if (!res.ok) continue;
      const data = await res.json();

      for (const item of (data.injuries || [])) {
        const athleteName = item.athlete?.displayName;
        const teamAbbr = item.athlete?.team?.abbreviation?.toUpperCase();
      }
    }
  }

  return processed;
}
```

```

const status = item.status; // "Out", "Questionable", "Probable", "Day-To
const note = item.detail || item.longComment || "";
const reportedAt = item.date ? new Date(item.date) : new Date();

if (!athleteName) continue;

// Match to player in DB by name + sport
const [player] = await db.select().from(playersTable)
  .where(and(eq(playersTable.fullName, athleteName), eq(playersTable.spor
  .limit(1);

if (!player) continue;

// Normalize status
const normalizedStatus = status.toLowerCase().includes("out") ? "out"
  : status.toLowerCase().includes("question") ? "gtd"
  : status.toLowerCase().includes("probable") ? "probable"
  : "active";

// Upsert injury record
await db.insert(injuriesTable).values({
  playerId: player.id,
  status: normalizedStatus,
  note,
  source: "espn",
  reportedAt,
}).onConflictDoUpdate({
  target: [injuriesTable.playerId],
  set: { status: normalizedStatus, note, reportedAt, updatedAt: new Date(
});

// Alert if player is in an active entry and status is OUT or GTD
if (normalizedStatus === "out" || normalizedStatus === "gtd") {
  await checkAndAlertActiveEntryInjury(player.id, normalizedStatus, athle
}

processed++;
}
} catch (e) {
  logger.error({ err: e, sport }, "Injury sync error");
}
}

return processed;
}

async function checkAndAlertActiveEntryInjury(

```

```

    playerId: number,
    status: string,
    playerName: string,
  ) {
    // Check if this player has an active (pending) entry pick
    const activePicks = await db.select()
      .from(entryPicksTable)
      .innerJoin(ppLinesTable, eq(entryPicksTable.ppLineId, ppLinesTable.id))
      .where(and(
        eq(ppLinesTable.playerId, playerId),
        eq(entryPicksTable.result, "pending"),
      ));

    if (activePicks.length > 0) {
      // Broadcast SSE alert
      broadcast("injury_alert", {
        playerName,
        status: status.toUpperCase(),
        message: `${playerName} is now listed ${status.toUpperCase()} - check your
          severity: status === "out" ? "critical" : "warning",
      });

      // Create a database alert
      await db.insert(alertsTable).values({
        type: "injury_update",
        severity: status === "out" ? "critical" : "warning",
        title: `${playerName} - ${status.toUpperCase()}`,
        message: `${playerName} is listed ${status.toUpperCase()} and appears in on
      });
    }
  }
}

```

Cron schedule update — add injury sync every 20 minutes:

```

// In cron.ts, replace the stub:
cron.schedule("* /20 * * * *", () =>
  logPull("espn", "injuries", syncInjuriesImpl)
);

```

Frontend — update Injuries & News page:

- Remove the "auto-sync not connected" empty state message
- Show real injury data from the database
- Color code: OUT = red border, GTD = amber border, Probable/Active = green border

- Sort by severity (OUT first, then GTD, then Probable)
- Show timestamp of when last updated
- Add a pulsing red dot in the sidebar nav next to “Injuries & News” when any OUT or GTD player is in an active entry

In the SSE listener in App.tsx — add injury_alert handler:

```
es.addEventListener("injury_alert", (e) => {
  const data = JSON.parse(e.data);
  toast({
    title: `⚠️ ${data.playerName} — ${data.status}`,
    description: data.message,
    variant: data.severity === "critical" ? "destructive" : "default",
    duration: 15000,
  });
});
```

Acceptance: Injuries & News page shows real player statuses. When a player in an active entry is listed OUT, a toast notification fires and the Injuries nav item shows a red dot.

PHASE 4: Underdog Fantasy Line Comparison (Week 2)

Adds a second major pick'em platform to your market comparison. When PP has 26.5 and Underdog has 28.5, that gap is real edge — and you now see it.

How Underdog's API works: The unofficial endpoint returns all pick'em lines in a structure similar to PrizePicks. No authentication required.

```
// artifacts/api-server/src/lib/sync/underdog.ts

const UD_BASE = "https://api.underdogfantasy.com/beta/v6";

export async function syncUnderdogLines(): Promise<number> {
  const res = await fetch(`${UD_BASE}/over_under_lines`, {
    headers: {
      "User-Agent": "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/",
      "Accept": "application/json",
      "Referer": "https://underdogfantasy.com",
      "Origin": "https://underdogfantasy.com",
    }
  });
  if (!res.ok) throw new Error(`Underdog API ${res.status}`);
  const data = await res.json();
```



```

// Underdog response: data.over_under_lines[] with data.appearances[] and data.
const playerMap: Record<string, any> = {};
for (const p of (data.players || [])) playerMap[p.id] = p;

const appearanceMap: Record<string, any> = {};
for (const a of (data.appearances || [])) appearanceMap[a.id] = a;

let processed = 0;

for (const line of (data.over_under_lines || [])) {
  try {
    const appearance = appearanceMap[line.over_under?.appearance_id];
    if (!appearance) continue;

    const player = playerMap[appearance.player_id];
    if (!player) continue;

    const playerName = `${player.first_name} ${player.last_name}`;
    const statType = line.over_under?.stat_value; // e.g. "Points"
    const lineValue = parseFloat(line.over_under?.value || "0");
    const sport = appearance.sport_id?.toUpperCase();

    if (!playerName || !statType || !lineValue) continue;

    // Match to our player by name + sport
    const [dbPlayer] = await db.select().from(playersTable)
      .where(and(eq(playersTable.fullName, playerName), eq(playersTable.sport,
        .limit(1);

    if (!dbPlayer) continue;

    // Find matching PP line
    const [ppLine] = await db.select().from(ppLinesTable)
      .where(and(
        eq(ppLinesTable.playerId, dbPlayer.id),
        eq(ppLinesTable.statType, statType),
        eq(ppLinesTable.isActive, true),
      )).limit(1);

    // Store as external line – reuses existing infrastructure
    await db.insert(externalLinesTable).values({
      playerId: dbPlayer.id,
      ppLineId: ppLine?.id ?? null,
      bookName: "underdog",
      statType,
      lineValue: lineValue.toString(),
    });
  } catch {
    // ...
  }
}

```

```

        capturedAt: new Date(),
    }).onConflictDoUpdate({
        target: [externalLinesTable.ppLineId, externalLinesTable.bookName],
        set: { lineValue: lineValue.toString(), capturedAt: new Date() }
    });

    processed++;
} catch { /* non-fatal */ }
}

// Trigger prop score recalculation to incorporate Underdog into true edge
await recalcPropScores();
return processed;
}

```

Add to sync routes and Force Sync:

```

// In sync.ts – add to /sync/all jobs array:
{ name: "underdog", provider: "underdog-fantasy", fn: syncUnderdogLines },

// New individual route:
router.post("/sync/underdog", async (req, res) => {
    await runSync("underdog-fantasy", "underdog", syncUnderdogLines, res);
});

```

Add to cron — every 15 minutes:

```

cron.schedule("*/15 * * * *", () =>
    logPull("underdog-fantasy", "underdog", syncUnderdogLines)
);

```

Add to environment variables:

```

UD_API_BASE=https://api.underdogfantasy.com/beta/v6

```

Frontend — market-intel display:

The `externalLinesTable` already stores by `bookName`. The market-intel route already aggregates all books. Underdog lines will automatically appear in `bookLines` as `{ underdog: 28.5, draftkings: 28.5, fanduel: 28.0, espnbet: 28.0 }`.

Update the market-intel route to show Underdog specifically in the response:

```

// In market-intel.ts return object, add:

```

```

underdogLine: bookLines["underdog"] ?? null,
ppVsUnderdog: bookLines["underdog"]
  ?  $(-(ppLine - bookLines["underdog"]) / bookLines["underdog"] * 100).toFixed(1)$ 
  : null,

```

In the Slate Board, add a “UD” column showing the Underdog line alongside the PP line. When PP is lower than Underdog (OVER value), highlight it green. When PP is higher than Underdog (UNDER value or skip), highlight it amber.

Acceptance: Slate Board shows Underdog lines for matched players. A prop where PP has 24.5 and Underdog has 27.5 shows “+12.2% vs UD” in green.

PHASE 5: Sharp Money Heuristic (Week 3)

You’re already logging every line move. This phase builds the analysis layer on top of data you’ve been collecting since the app launched.

How to detect sharp money without buying data:

Sharp bettors move lines before the public. DraftKings is typically the market-maker — it moves first when sharp money comes in. When FanDuel follows within 30 minutes in the same direction, that’s a strong signal of informed betting. When the line moves *against* the direction of heavy public action (reverse line movement), that’s even stronger.

```

// artifacts/api-server/src/lib/analysis/sharp-money.ts

import { db } from "@workspace/db";
import { lineMoveEventsTable, propScoresTable } from "@workspace/db/schema";
import { eq, asc } from "drizzle-orm";

export interface SharpSignal {
  isSharp: boolean;
  isReverseLineMove: boolean;
  confidence: "high" | "medium" | "low" | "none";
  direction: "more" | "less" | null; // which direction sharp money is on
  reason: string;
}

export async function detectSharpSignal(ppLineId: number): Promise<SharpSignal> {
  const moves = await db.select().from(lineMoveEventsTable)
    .where(eq(lineMoveEventsTable.ppLineId, ppLineId))
    .orderBy(asc(lineMoveEventsTable.capturedAt));

  if (moves.length < 2) {

```

```

    return { isSharp: false, isReverseLineMove: false, confidence: "none", direct
}

// Find DraftKings moves (market-maker)
const dkMoves = moves.filter(m => m.bookName === "draftkings");
const fdMoves = moves.filter(m => m.bookName === "fanduel");
const espnMoves = moves.filter(m => m.bookName === "espnbet");
const udMoves = moves.filter(m => m.bookName === "underdog");

const firstDkMove = dkMoves[0];
if (!firstDkMove) {
    return { isSharp: false, isReverseLineMove: false, confidence: "none", direct
}

const dkDirection = firstDkMove.moveDirection as "up" | "down";
const dkTime = firstDkMove.capturedAt;

// Count books that followed DK in same direction within 30 minutes
const followers = [...fdMoves, ...espnMoves, ...udMoves].filter(m => {
    const timeDiff = (new Date(m.capturedAt).getTime() - new Date(dkTime).getTime
    return timeDiff > 0 && timeDiff < 30 && m.moveDirection === dkDirection;
});

// DK moved first + 2+ books followed = high confidence sharp
if (followers.length >= 2) {
    return {
        isSharp: true,
        isReverseLineMove: false,
        confidence: "high",
        direction: dkDirection === "up" ? "less" : "more", // line went up → sharps
        reason: `DK led, ${followers.length} books followed within 30 min`,
    };
}

// DK moved first + 1 book followed = medium confidence
if (followers.length === 1) {
    return {
        isSharp: true,
        isReverseLineMove: false,
        confidence: "medium",
        direction: dkDirection === "up" ? "less" : "more",
        reason: `DK-led move, FD/ESPN confirmed`,
    };
}

// DK moved alone – could be sharp, could be news
return {

```

```

    isSharp: false,
    isReverseLineMove: false,
    confidence: "low",
    direction: null,
    reason: "DK moved, no confirmation from other books",
  };
}

// Run on all active lines after each odds sync
export async function updateAllSharpSignals(): Promise<void> {
  const lines = await db.select({ id: propScoresTable.ppLineId })
    .from(propScoresTable);

  for (const { id } of lines) {
    if (!id) continue;
    const signal = await detectSharpSignal(id);
    // Store signal in prop_scores.reasoning jsonb field
    await db.update(propScoresTable)
      .set({
        reasoning: db.sql`reasoning || ${JSON.stringify({ sharpSignal: signal })}`
      })
      .where(eq(propScoresTable.ppLineId, id));
  }
}

```

Call after every odds sync:

```

// In external-odds.ts, after recalcPropScores():
import { updateAllSharpSignals } from "../analysis/sharp-money";
await updateAllSharpSignals();

```

Frontend — Slate Board:

Add a sharp money indicator column. Show:

- ⚡ with confidence color when sharp signal detected
- Direction label: "SHARP → UNDER" or "SHARP → OVER"
- Tooltip explaining: "DraftKings moved first, FanDuel confirmed within 18 min"

```

// In Slate Board table header:
<TableHead className="hidden lg:table-cell w-20 font-mono text-xs">Sharp</TableHead>

// In table row:
<TableCell className="hidden lg:table-cell">

```

```

{row.scoring?.sharpSignal?.isSharp ? (
  <Tooltip>
    <TooltipTrigger>
      <div className={`flex items-center gap-1 text-[10px] font-mono font-bold
        row.scoring.sharpSignal.confidence === "high" ? "text-violet-400" : "te
      }`} >
        ⚡ {row.scoring.sharpSignal.direction?.toUpperCase()}
      </div>
    </TooltipTrigger>
    <TooltipContent className="text-xs max-w-xs">
      <p>{row.scoring.sharpSignal.reason}</p>
      <p className="text-slate-400 mt-1">Confidence: {row.scoring.sharpSignal.c
    </TooltipContent>
  </Tooltip>
) : <span className="text-slate-700 text-[10px]">—</span>}
</TableCell>

```

Also add to “Why This Edge Exists” panel in Prop Detail Sheet: When

sharpSignal.isSharp = true , add a line: “Sharp money detected — DraftKings led, confirmed by [N] books within [X] minutes. Smart money appears to be on the [direction].”

Acceptance: Props with confirmed sharp action show the ⚡ badge. Props without it show “—”. Tooltip explains exactly what triggered it.

PHASE 6: Push Notifications (Week 3)

The highest value-per-dollar feature in the roadmap. Alerts you in real time when something that affects your picks happens — even if you’re not looking at the screen.

Setup — Twilio SMS (~\$5/month at normal usage):

```

TWILIO_ACCOUNT_SID=AC...    # from twilio.com console
TWILIO_AUTH_TOKEN=...
TWILIO_FROM_NUMBER=+1...    # your Twilio phone number

```

Add your phone number to user_settings:

```

// In user-settings schema, add:
phoneNumber: varchar("phone_number", { length: 20 }),
smsAlertsEnabled: boolean("sms_alerts_enabled").default(false),
smsAlertTypes: jsonb("sms_alert_types").default({
  newGoblin: true,          // new goblin line appears on the slate
  majorLineMove: true,      // any line moves 1+ point
  injuryInEntry: true,      // player in active entry is OUT or GTD

```

```

    lossLimitHit: true,          // daily loss limit reached
    syncFailure: false,         // sync job fails (off by default)
  }),

```

SMS service:

```

// artifacts/api-server/src/lib/notifications/sms.ts

import twilio from "twilio";

const client = process.env.TWILIO_ACCOUNT_SID
  ? twilio(process.env.TWILIO_ACCOUNT_SID, process.env.TWILIO_AUTH_TOKEN)
  : null;

export async function sendSMS(to: string, message: string): Promise<void> {
  if (!client || !process.env.TWILIO_FROM_NUMBER) {
    console.warn("SMS not configured - skipping notification");
    return;
  }
  await client.messages.create({
    body: message,
    from: process.env.TWILIO_FROM_NUMBER,
    to,
  });
}

export async function notifyUser(
  alertType: string,
  message: string,
): Promise<void> {
  // Fetch user settings
  const [settings] = await db.select().from(userSettingsTable).limit(1);
  if (!settings?.smsAlertsEnabled || !settings?.phoneNumber) return;

  const types = settings.smsAlertTypes as Record<string, boolean>;
  if (!types[alertType]) return;

  await sendSMS(settings.phoneNumber, `VibeMeGood: ${message}`);
}

```

Wire into existing event points:

```

// In syncPpLines() - when a new goblin line is found:
broadcastNewGoblin(playerName, statType, lineValue, sport);
await notifyUser("newGoblin",
  `New goblin: ${playerName} ${statType} ${lineValue} (${sport})`);

```

```

// In injury sync – when player in active entry is OUT:
broadcast("injury_alert", { playerName, status });
await notifyUser("injuryInEntry",
  `⚠️ ${playerName} is now ${status.toUpperCase()} – check your active entries imm

// In external-odds sync – when any line moves 1+ point:
if (Math.abs(moveSize) >= 1.0) {
  await notifyUser("majorLineMove",
    `${playerName} ${statType}: line moved ${moveSize > 0 ? "+" : ""}${moveSize}
}

// In loss limit check:
await notifyUser("lossLimitHit",
  `Daily loss limit of $$${limit} reached. No more entries today.`);

```

Settings page — add SMS section:

```

<div className="space-y-4">
  <div className="text-xs font-mono uppercase tracking-wider text-muted-foreground">
    SMS Alerts (Twilio)
  </div>
  <div className="flex items-center gap-3">
    <Input
      value={settings.phoneNumber ?? ""}
      onChange={e => onChange({ phoneNumber: e.target.value })}
      placeholder="+1 555 000 0000"
      className="font-mono text-sm"
    />
    <Switch
      checked={settings.smsAlertsEnabled}
      onCheckedChange={v => onChange({ smsAlertsEnabled: v })}
    />
  </div>
  {settings.smsAlertsEnabled && (
    <div className="space-y-2 pl-2 border-l-2 border-primary/20">
      {[
        { key: "newGoblin",      label: "New goblin line appears" },
        { key: "majorLineMove",  label: "Line moves 1+ point" },
        { key: "injuryInEntry",  label: "Player in active entry is OUT/GTD" },
        { key: "lossLimitHit",   label: "Daily loss limit hit" },
      ].map(({ key, label }) => (
        <div key={key} className="flex items-center justify-between">
          <span className="text-sm text-muted-foreground">{label}</span>
          <Switch
            checked={settings.smsAlertTypes?.[key] ?? false}

```



```

        onChange={v => onChange({
          smsAlertTypes: { ...settings.smsAlertTypes, [key]: v }
        })}
      />
    </div>
  )}
</div>
)}
</div>

```

Acceptance: Enter a phone number in Settings, enable SMS alerts. When a new goblin line is detected in the next PP sync, receive an SMS within 2 minutes. When a player in an active entry is scratched, receive an SMS immediately.

PHASE 7: Variance Intelligence Engine (Week 4)

Read: [VibeMeGood-VarianceEngine.md](#)

Full spec with all implementation details in that file. Summary of what it adds:

- Master toggle in Settings: Variance Intelligence ON/OFF
- When OFF: app behaves identically to today
- When ON: every prop gets scored for fatigue, blowout risk, usage trends, matchup depth
- “Why This Edge Exists” panel appears on every prop detail sheet
- Volatility badges (STABLE / ELEVATED / VOLATILE) on Slate Board rows
- Optimizer gets new modes: Stable Grind, Ceiling Hunter
- EV modifier applied from validated signals only (max $\pm 15\%$)
- Experimental Lab: sandboxed curiosity features behind separate confirmation

Data sources used — all from existing integrations:

- Back-to-back: computed from game schedule dates already in DB
- Blowout probability: derived from spread + total already pulled from Odds API
- Usage trend: computed from player_game_logs already being stored
- Matchup: matchup_history table already exists
- Venue/altitude: hardcoded lookup table (in the spec)

Acceptance: 10-point acceptance test is in `VibeMeGood-VarianceEngine.md`.

PHASE 8: Polish Pass (Week 4)

After all phases above are complete:

1. Performance review — CLV as primary metric

- 7-day, 30-day, 90-day CLV averages at the top
- CLV trend chart as the primary visualization
- Behavioral discipline messages:
 - Losing streak + positive CLV: "Your process is sound. This is variance."
 - Winning streak + negative CLV: "Recent wins may be luck. Reassess."
- Win rate and P&L moved to secondary position

2. AI Analyst context upgrade

- Inject sharp money signals into system prompt when available
- Inject Underdog vs PP comparison into context
- Inject injury status for every player being discussed
- Inject variance scores for props being analyzed

3. Startup health check

- On server start, print a status table showing which data sources are active:

PrizePicks sync:	✓ last run 4 min ago (247 lines)
External odds:	✓ last run 18 min ago (183 lines)
Underdog:	✓ last run 14 min ago (201 lines)
Injuries:	✓ last run 19 min ago (34 players)
Projections:	✓ computed at startup (218 players)
Variance scores:	✓ computed at startup (218 props)
SMS alerts:	✓ Twilio configured

ENVIRONMENT VARIABLES — COMPLETE LIST

Add all of these to Replit Secrets before running anything:

```
# Core (already configured)
DATABASE_URL=
CLERK_SECRET_KEY=
CLERK_PUBLISHABLE_KEY=
STRIPE_SECRET_KEY=
ANTHROPIC_API_KEY=

# Data APIs (all free tiers)
ODDS_API_KEY=          # the-odds-api.com - 500 req/month free
OPENWEATHER_API_KEY=   # openweathermap.org - 1000 req/day free

# New - Phase 6 (Twilio SMS)
TWILIO_ACCOUNT_SID=    # twilio.com - ~$5/month usage
TWILIO_AUTH_TOKEN=
TWILIO_FROM_NUMBER=    # your Twilio phone number

# Base URLs (do not change)
PP_API_BASE=https://api.prizepicks.com
ODDS_API_BASE=https://api.the-odds-api.com/v4
UD_API_BASE=https://api.underdogfantasy.com/beta/v6
NBA_STATS_BASE=https://stats.nba.com/stats
NHL_API_BASE=https://api.nhle.com/stats/rest/en
MLB_STATS_BASE=https://statsapi.mlb.com/api/v1
ESPN_NFL_BASE=https://site.api.espn.com/apis/site/v2/sports/football/nfl
ESPN_INJURIES_BASE=https://site.api.espn.com/apis/site/v2/sports
OW_BASE=https://api.openweathermap.org/data/2.5
```

FINAL ACCEPTANCE TEST

The build is complete when all of the following pass:

```
CORE (from original spec)
[ ] Force Sync shows PP + Underdog + external odds + projections all completing
[ ] Slate Board readable on 390px phone screen
[ ] 5+ NBA props show OUR PROJ with confidence badge
[ ] Optimizer Goblin Hunter 4-pick returns all Goblin OVER picks with EV shown
[ ] Mark pick HIT with closing line → CLV record in CLV Tracker
[ ] GET /api/debug/calibration returns sampleSize and source label
[ ] Loss limit blocks entry after limit hit
[ ] User Guide loads and is searchable
[ ] DB transactions confirmed: marking a pick updates all tables or none

INJURIES (Phase 3)
```

- [] Injuries page shows real player statuses from ESPN
- [] OUT/GTD player in active entry triggers a toast notification
- [] Sidebar shows injury indicator when active entry has GTD/OUT player

UNDERDOG (Phase 4)

- [] Slate Board shows Underdog line for matched players
- [] Market Intel shows PP vs Underdog gap where available
- [] bookLines object in market-intel response includes "underdog" key

SHARP MONEY (Phase 5)

- [] After DK and FD both move the same direction in sequence, ⚡ badge appears
- [] Prop Detail Sheet explains sharp signal in "Why This Edge Exists" panel
- [] Props with no line movement show "-" in sharp column

SMS ALERTS (Phase 6)

- [] Enter phone number in Settings, enable New Goblin alerts
- [] Trigger a PP sync that finds a new goblin line
- [] Receive SMS within 2 minutes
- [] Disabling the toggle stops SMS without needing to remove the phone number

VARIANCE ENGINE (Phase 7)

- [] Toggle OFF: Slate Board looks identical to pre-variance build
- [] Toggle ON: volatility badges visible on every prop row
- [] Prop with confirmed back-to-back shows "back_to_back" warning
- [] Experimental Lab requires confirmation dialog before enabling
- [] Narrative signals appear in Why This Edge panel but show 0 EV contribution

PHOTOS (Phase 2)

- [] Every player row shows circular photo or colored initials
- [] Failed image URL falls back to initials, no broken icon

COST SUMMARY

Service	Monthly Cost
Replit Pro	\$25
Twilio SMS (~200 messages)	~\$5
Anthropic Claude (personal use)	\$5–10
Everything else	\$0
Total	~\$35–40/month

